

LaunchMind Blueprint v2.0

Engineering Implementation Plan — Architecture Baseline v1.0 → Production Vijay Tomar · July 2026 ·
Built with Claude Code + claude-sonnet-4-6

Source of truth: docs/architecture-baseline-v1.md **Current build:** Phase 5 Week 20 —
COMPLETE (see CLAUDE.md Section 11) **This document:** Maps current implementation → target
architecture · Defines every gap · Plans phases 6–9

Part 1 — Architecture Delta

What exists today vs. what the baseline requires

Baseline Concept	Current Implementation	Status	
Growth Brain	products.confirmed_icp + products.brand_voice_profile + strategy output	Partial	No versioning, no confidence scores, no positioning/differentiators schema
Marketing Memory	content_learnings + weekly_briefs + playbook_signals	Partial	No Founder/Brand/Campaign/Customer/C memory taxonomy
Knowledge Graph	Not built	Missing	Entities + relationships are stored flat
Context Engine	Ad-hoc context assembly in contentService.ts	Partial	No unified pre-request context builder differently
Agent Platform	Service functions (not agents) in contentService.ts, strategyService.ts	Partial	No named agents, no agent registry, no agent memory
Mission Orchestrator	BullMQ workers (intakeWorker, briefWorker)	Partial	Handles jobs, not missions; no mission retry/recovery per mission
Recommendation Engine	playbook_signals lookup (partial)	Partial	No prioritisation logic, no opportunity
Decision Engine	Mixed into AI prompts	Missing	Business rules not separated from AI; no rules engine
Intelligence Network	playbook_signals (52 seed rows)	Partial	No live signal ingestion, no anonymisation
Execution Platform	Campaigns + content assets + channels	Partial	No experiments, no calendar, no unified execution
Morning Brief	weekly_briefs (Sunday cron)	Partial	Daily cadence not implemented; no project objective framing
Missions	Not built	Missing	No mission concept, no mission state
Approvals	campaigns.approved_at gate	Partial	Only covers campaigns; no unified approval

Ask LaunchMind	Not built	Missing	No conversational AI interface
Opportunities	Not built	Missing	No opportunity discovery or scoring
Experiments	Not built	Missing	No A/B experiment tracking
Calendar	Not built	Missing	No content/campaign calendar view
Results	campaign_metrics table	Partial	No unified results view; no ROAS/outcome

Part 2 — Navigation: Current → Target

Current sidebar navigation (Phase 5)

Dashboard
Products
Campaigns
Briefs
Insights (Builder/Studio)
Workspaces (Studio)
Channels
Billing
Settings

Target navigation (Architecture Baseline §6)

Home
└─ Morning Brief
└─ Opportunities
└─ Ask LaunchMind
Missions
└─ Active missions
└─ Approvals
Results
Execution
└─ Content Studio
└─ Campaigns
└─ Experiments
└─ Calendar
Intelligence
└─ Growth Brain
└─ Market Intelligence
└─ Reviews
└─ Ideas Inbox
└─ Timeline
Manage

- Settings
- Billing

Next.js route map (target)

Nav Item	Route	Notes
Home	/dashboard	Replaces current dashboard
Morning Brief	/dashboard/brief	Replaces /dashboard/briefs — daily, not weekly
Opportunities	/dashboard/opportunities	New
Ask LaunchMind	/dashboard/ask	New — conversational interface
Missions (list)	/dashboard/missions	New
Approvals	/dashboard/approvals	New — unified approval queue
Results	/dashboard/results	Replaces /dashboard/insights
Content Studio	/dashboard/content	Replaces /dashboard/briefs asset view
Campaigns	/dashboard/campaigns	Existing — extend
Experiments	/dashboard/experiments	New
Calendar	/dashboard/calendar	New
Growth Brain	/dashboard/intelligence/growth-brain	New
Market Intelligence	/dashboard/intelligence/market	New
Reviews	/dashboard/intelligence/reviews	New
Ideas Inbox	/dashboard/intelligence/ideas	New
Timeline	/dashboard/intelligence/timeline	New
Settings	/dashboard/settings	Existing — extend
Billing	/dashboard/billing	Existing — extend

Part 3 — Intelligence Layer Specifications

3.1 Growth Brain

What it is: The living, versioned strategy for every product. The single source of truth for what LaunchMind knows about a product's positioning, ICP, goals, and confidence in each channel.

Current data: `products.confirmed_icp` (JSONB) + `products.brand_voice_profile` (JSONB)

Gap: Flat, unversioned, no confidence scores, no evidence, no timeline.

New table: `growth_brain`

```
CREATE TABLE growth_brain (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id  UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  founder_id  UUID NOT NULL REFERENCES founders(id),
  version     INTEGER NOT NULL DEFAULT 1,
  positioning TEXT,
  icp         JSONB,          -- target user, pain points, geography,
  psychographics
  differentiators JSONB,      -- MOAT, unique features, evidence
  messaging      JSONB,      -- primary message, proof points, tone
  business_objectives JSONB,  -- goal type, target metric, timeline
  known_risks    JSONB,
  channel_confidence JSONB,   -- per-channel: score (0-1), evidence,
  last_updated
  confidence_score NUMERIC(4,3), -- overall 0-1
  evidence         JSONB,      -- supporting data points
  is_current       BOOLEAN NOT NULL DEFAULT true,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Only one current version per product
CREATE UNIQUE INDEX growth_brain_current ON growth_brain(product_id) WHERE
is_current = true;
ALTER TABLE growth_brain ENABLE ROW LEVEL SECURITY;
CREATE POLICY "gb_owner" ON growth_brain USING (founder_id = auth.uid());
```

Migration: Extract `confirmed_icp` + `brand_voice_profile` into first `growth_brain` row on product confirm. Keep existing columns for backward compat.

3.2 Marketing Memory

What it is: Persistent categorised learning store. Every signal (approval, rejection, winner, loser, review, campaign result) enriches memory by category.

Current data: `content_learnings` (partial) + `weekly_briefs.what_worked` + `weekly_briefs.what_to_kill`

Gap: No memory taxonomy; learnings are per-asset, not categorised by memory type.

New table: `marketing_memory`

```
CREATE TABLE marketing_memory (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id  UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  founder_id  UUID NOT NULL REFERENCES founders(id),
  memory_type TEXT NOT NULL CHECK (memory_type IN (
    'founder', 'brand', 'product', 'campaign',
    'customer', 'review', 'competitor', 'market',
    'experiment', 'seasonality'
  )),
  key         TEXT NOT NULL,    -- what is remembered
```

```

value          JSONB NOT NULL,      -- structured memory value
confidence     NUMERIC(4,3),        -- 0-1
source         TEXT,                -- what created this memory
source_id      UUID,                -- reference to source row
supersedes_id  UUID REFERENCES marketing_memory(id),
embedding      VECTOR(1536),
expires_at     TIMESTAMPTZ,
created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);
ALTER TABLE marketing_memory ENABLE ROW LEVEL SECURITY;
CREATE POLICY "mm_owner" ON marketing_memory USING (founder_id = auth.uid());
CREATE INDEX mm_product_type ON marketing_memory(product_id, memory_type);

```

Migration from `content_learnings` : One-time migration job converts existing `content_learnings` rows to `marketing_memory` with appropriate `memory_type` .

3.3 Knowledge Graph

What it is: Entity-relationship model of everything LaunchMind knows about a product's ecosystem. Relationships are first-class citizens.

New tables: `kg_entities` + `kg_relationships`

```

CREATE TABLE kg_entities (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id  UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  founder_id  UUID NOT NULL REFERENCES founders(id),
  entity_type TEXT NOT NULL CHECK (entity_type IN (
    'product', 'feature', 'persona', 'problem',
    'competitor', 'campaign', 'channel', 'creative',
    'experiment', 'review', 'pricing', 'market'
  )),
  name        TEXT NOT NULL,
  properties   JSONB,
  embedding    VECTOR(1536),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE kg_relationships (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id  UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  founder_id  UUID NOT NULL REFERENCES founders(id),
  from_entity_id  UUID NOT NULL REFERENCES kg_entities(id) ON DELETE CASCADE,
  to_entity_id    UUID NOT NULL REFERENCES kg_entities(id) ON DELETE CASCADE,
  relationship    TEXT NOT NULL, -- e.g. 'solves', 'competes_with', 'targets',
  'performed_on'
  properties      JSONB,
  confidence      NUMERIC(4,3),
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```
ALTER TABLE kg_entities ENABLE ROW LEVEL SECURITY;
ALTER TABLE kg_relationships ENABLE ROW LEVEL SECURITY;
CREATE POLICY "kg_entities_owner" ON kg_entities USING (founder_id = auth.uid());
CREATE POLICY "kg_relationships_owner" ON kg_relationships USING (founder_id = auth.uid());
```

Population strategy: Knowledge Graph is built incrementally from intake data (competitors, features from description, personas from ICP). A `KnowledgeGraphAgent` runs after intake and after each weekly brief.

3.4 Context Engine

What it is: A single service that assembles the complete AI context before every request. No AI call bypasses Context Engine.

Current gap: Each service (`contentService` , `strategyService` , `playbookService`) assembles its own ad-hoc context.

New service: `backend/src/services/contextEngine.ts`

```
export interface AIContext {
  growthBrain: GrowthBrainVersion;
  marketingMemory: MarketingMemorySlice;
  knowledgeGraph: KGSummary;
  timeline: TimelineEntry[];
  experiments: ExperimentSummary[];
  recentReviews: ReviewSummary[];
  competitorDelta: CompetitorDelta[];
  results: ResultsSummary;
  intelligenceSignals: PlaybookSignal[];
  founderPreferences: FounderPrefs;
  brandVoice: BrandVoiceProfile;
  currentMission?: Mission;
  budget: BudgetContext;
}

export async function assembleContext(
  productId: string,
  founderId: string,
  opts?: { missionId?: string; depth?: 'minimal' | 'standard' | 'full' }
): Promise<AIContext>
```

Rule: Every AI call in the codebase is migrated to call `assembleContext()` first. Context is passed as the first argument to all AI service functions.

3.5 Mission Orchestrator

What it is: A state machine that coordinates agents, approvals, publishing, experiments, and learning for a single business objective.

New table: `missions`

```

CREATE TABLE missions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id  UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  founder_id  UUID NOT NULL REFERENCES founders(id),
  title       TEXT NOT NULL,
  objective   TEXT NOT NULL, -- business outcome, not channel
  objective_type TEXT NOT NULL CHECK (objective_type IN (
    'increase_installs','reduce_cpi','improve_aso',
    'launch_market','increase_ratings','recover_churn',
    'increase_revenue','other'
  )),
  target_metric JSONB,      -- { metric: 'installs', target: 1000, timeline:
                             '30d' }
  status       TEXT NOT NULL DEFAULT 'planning' CHECK (status IN (
    'planning','active','awaiting_approval',
    'executing','paused','completed','cancelled'
  )),
  agent_plan   JSONB,      -- planned agent steps
  agent_state  JSONB,      -- current agent execution state
  approvals   JSONB,      -- approval checkpoints and outcomes
  results      JSONB,      -- mission outcomes
  ai_reasoning TEXT,      -- why LaunchMind chose this mission
  confidence   NUMERIC(4,3),
  growth_brain_version_id UUID REFERENCES growth_brain(id),
  started_at   TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
ALTER TABLE missions ENABLE ROW LEVEL SECURITY;
CREATE POLICY "missions_owner" ON missions USING (founder_id = auth.uid());

```

3.6 Recommendation Engine

What it is: Produces ranked opportunities and missions from signals, growth brain, memory, budget, and risk.

New table: `opportunities`

```

CREATE TABLE opportunities (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id  UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  founder_id  UUID NOT NULL REFERENCES founders(id),
  title       TEXT NOT NULL,
  description  TEXT,
  opportunity_type TEXT NOT NULL, --
  'quick_win','strategic','defensive','experimental'
  evidence    JSONB,
  confidence   NUMERIC(4,3),
  expected_impact JSONB,      -- { metric, delta, timeline }
  risk        TEXT,

```

```

effort          TEXT,          -- 'low', 'medium', 'high'
recommended_mission JSONB,
status          TEXT NOT NULL DEFAULT 'open' CHECK (status IN
                ('open', 'accepted', 'dismissed', 'expired')),
expires_at      TIMESTAMPTZ,
created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
ALTER TABLE opportunities ENABLE ROW LEVEL SECURITY;
CREATE POLICY "opp_owner" ON opportunities USING (founder_id = auth.uid());

```

3.7 Experiments

What it is: Structured A/B experiment tracking for content, campaigns, and channel strategies.

New table: `experiments`

```

CREATE TABLE experiments (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id  UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  founder_id  UUID NOT NULL REFERENCES founders(id),
  mission_id  UUID REFERENCES missions(id),
  title       TEXT NOT NULL,
  hypothesis  TEXT,
  variant_a   JSONB NOT NULL,
  variant_b   JSONB NOT NULL,
  metric      TEXT NOT NULL,
  status      TEXT NOT NULL DEFAULT 'draft' CHECK (status IN
                ('draft', 'running', 'paused', 'concluded')),
  winner      TEXT,          -- 'a', 'b', or 'inconclusive'
  result_data JSONB,
  started_at  TIMESTAMPTZ,
  concluded_at TIMESTAMPTZ,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
ALTER TABLE experiments ENABLE ROW LEVEL SECURITY;
CREATE POLICY "exp_owner" ON experiments USING (founder_id = auth.uid());

```

Part 4 — Agent Registry

Each agent is a named service with a single responsibility. All agents communicate through Mission Orchestrator, never directly.

Agent	Responsibility	Current Equivalent	Status
ResearchAgent	Gather market intel, competitor data, review signals	scraperWorker, reviewAnalysis	Refactor
StrategyAgent	Generate Growth Brain versions, mission plans	strategyService.ts	Refactor

PlanningAgent	Decompose missions into agent steps	—	New
ContentAgent	Generate all 22 asset types	contentService.ts	Refactor
CreativeAgent	Images, video, voice	replicateClient, creatomateClient, elevenLabsClient	Refactor
CampaignAgent	Meta/Google campaign creation	Campaign routes	Refactor
PublishingAgent	Cross-channel publishing with approval gates	Campaign approve routes	Refactor
OptimizationAgent	Analyse results, suggest bid/budget changes	—	New
LearningAgent	Update Marketing Memory, Growth Brain from results	briefWorker (partial)	Refactor
ReportingAgent	Produce Morning Brief, Results summaries	briefWorker (partial)	Refactor
MemoryAgent	Knowledge Graph maintenance, memory hygiene	—	New
BenchmarkAgent	Pull Intelligence Network signals for context	playbook_signals	Refactor

Rule: Every agent takes (context: AIContext, payload: AgentPayload) => Promise<AgentResult> . No agent calls Claude directly — all Claude calls go through aiClient.ts .

Part 5 — Decision Engine

Separates business rules (hard logic) from AI reasoning (soft judgement).

Examples of business rules (never delegated to AI):

- Budget cap enforcement (422 if over cap)
- Approval gate enforcement (approved_at must be non-null)
- Plan-tier feature gating (Studio = workspaces, Builder = playbook insights)
- Token balance enforcement
- MFA requirement
- Experiment minimum runtime (no conclusion before 7 days)

Examples of AI reasoning (Context Engine + agent):

- Which mission to recommend
- Which channel to prioritise
- What hook angle to use
- Whether a creative is good enough to approve

New service: backend/src/services/decisionEngine.ts

```
export function checkBudgetCap(campaignId: string, proposed: number): Promise<void>
export function checkApprovalGate(resourceType: string, resourceId: string):
Promise<void>
export function checkPlanFeature(founderId: string, feature: string): Promise<void>
export function checkTokenBalance(founderId: string, cost: number): Promise<void>
export function checkExperimentRuntime(experimentId: string): Promise<void>
```

All existing inline guard clauses in route handlers are migrated here.

Part 6 — Implementation Phases

Phase 6 — Foundation Intelligence (Weeks 21–24)

Goal: Growth Brain, Context Engine, Marketing Memory, new navigation shell.

Backend (backend-first rule applies to every item):

- Migration 031: `growth_brain` table
- Migration 032: `marketing_memory` table
- Migration 033: `missions` table
- Migration 034: `opportunities` table
- `services/contextEngine.ts` — `assembleContext()` with depth levels
- `services/growthBrainService.ts` — create/version/retrieve Growth Brain
- `services/marketingMemoryService.ts` — write/read by `memory_type`
- Migrate `strategyService.ts` to use `assembleContext()` as first step
- Migrate `contentService.ts` to use `assembleContext()` as first step
- Routes: `GET/PUT /growth-brain/:productId` · `GET /marketing-memory/:productId`

Frontend:

- New navigation shell (matches Architecture Baseline §6)
- `/dashboard` — new Home with Morning Brief + Opportunities (stubs initially)
- `/dashboard/missions` — mission list
- `/dashboard/intelligence/growth-brain` — Growth Brain viewer (read-only phase 6)

Definition of done:

- Every AI call routes through `assembleContext()`
 - Growth Brain created on product confirm (replaces raw `confirmed_icp` as primary AI context)
 - `tsc --noEmit`: 0 errors
-

Phase 7 — Mission Engine (Weeks 25–28)

Goal: Mission Orchestrator, Recommendation Engine, Approvals queue.

Backend:

- Migration 035: `experiments` table
- Migration 036: `kg_entities` + `kg_relationships` tables
- `services/missionOrchestrator.ts` — mission state machine
- `services/recommendationEngine.ts` — opportunity scoring
- `services/decisionEngine.ts` — business rule extraction

- BullMQ: `missionWorker.ts` — replaces ad-hoc job handling
- Routes: `POST /missions` · `GET /missions` · `POST /missions/:id/approve` · `GET /opportunities`

Frontend:

- `/dashboard/missions` — full mission CRUD + status tracking
- `/dashboard/approvals` — unified approval queue (campaigns + content + videos + experiments)
- `/dashboard/opportunities` — opportunity cards with accept/dismiss

Definition of done:

- Missions drive all content and campaign creation (not manual trigger)
- Unified approval queue replaces campaign-only approval flow

Phase 8 — Agent Platform (Weeks 29–32)

Goal: Named agents, Knowledge Graph, Learning Agent.

Backend:

- `services/agents/` directory — one file per agent
- `services/agents/index.ts` — agent registry
- `services/agents/contentAgent.ts` — refactored from `contentService.ts`
- `services/agents/learningAgent.ts` — Marketing Memory updater
- `services/agents/memoryAgent.ts` — Knowledge Graph maintenance
- `services/agents/benchmarkAgent.ts` — Intelligence Network signals
- Knowledge Graph population from intake data
- All agents take `(context: AIContext, payload)` signature

Frontend:

- Growth Brain viewer becomes editable (positioning, objectives, risks)
- `/dashboard/intelligence/timeline` — chronological product history
- `/dashboard/intelligence/market` — competitor + market signals

Definition of done:

- All 12 agents implemented and registered
- Knowledge Graph populated for all existing products
- `tsc --noEmit`: 0 errors

Phase 9 — Full Product (Weeks 33–36)

Goal: Ask LaunchMind, Experiments, Calendar, Results, Morning Brief as daily cadence.

Backend:

- `services/askService.ts` — conversational AI with full context engine
- Experiments end-to-end (create, run, conclude, apply learnings)
- Calendar scheduling logic
- Results attribution (installs/revenue traceable to mission)
- Morning Brief → daily cadence (replaces Sunday-only cron)
- Intelligence Network — live signal ingestion pipeline

Frontend:

- /dashboard/ask — Ask LaunchMind conversational UI
- /dashboard/experiments — A/B experiment builder + results
- /dashboard/calendar — content + campaign calendar
- /dashboard/results — unified results with attribution
- /dashboard/intelligence/reviews — review monitoring + response drafts
- /dashboard/intelligence/ideas — Ideas Inbox

Definition of done:

- Founder can state a business objective → LaunchMind creates a mission → executes → reports results
- No founder needs to think about channels, agents, or prompts
- E2E test: objective → mission → approval → execution → results

Part 7 — Database Migration Index (Phases 6–9)

Migration	Description	Phase
031	growth_brain table (versioned strategy)	6
032	marketing_memory table (10 memory types)	6
033	missions table	7
034	opportunities table	7
035	experiments table	8
036	kg_entities + kg_relationships tables	8
037	morning_briefs daily table (replaces weekly_briefs cadence)	9
038	results attribution table	9
039	intelligence_signals live ingestion table	9

Rule: All migrations additive only. Existing tables (products , campaigns , weekly_briefs , content_assets) are extended, not replaced.

Part 8 — ADR Index (Architecture Decision Records)

Every new architectural component requires an ADR. Stored in docs/adr/ .

ADR	Decision	Status
ADR-001	Context Engine: assemble-before-request pattern	Proposed
ADR-002	Growth Brain versioning: immutable rows + is_current flag	Proposed
ADR-003	Marketing Memory: 10-type taxonomy vs. flat learning store	Proposed
ADR-004	Agent communication: Mission Orchestrator as sole broker	Proposed
ADR-005	Knowledge Graph: pgvector relationships vs. dedicated graph DB	Proposed

ADR-006	Decision Engine: business rules separated from AI prompts	Proposed
ADR-007	Morning Brief: daily cadence replacing Sunday-only cron	Proposed
ADR-008	Navigation refactor: URL stability during migration	Proposed

Part 9 — Responsive + Visual Gaps (Existing Pages, Immediate)

Before Phase 6 begins, four existing dashboard pages have known responsive and visual polish gaps. These are tracked in the active plan and must be resolved first.

Page	Gap	Fix
campaigns/page.tsx	Channel icons have background box (reference shows bare icon)	Replace <ChannelIcon> with inline icon in table rows
briefs/page.tsx	2-col layout hardcoded (gridTemplateColumns: '1fr 1fr')	grid grid-cols-1 xl:grid-cols-2
channels/page.tsx	Lock icon grey not sage; connected cards missing sage border	color: var(--sage) on lock; borderColor: var(--sage-b) on connected card
billing/page.tsx	4-col plan grid not responsive; topbar padding fixed 32px	grid grid-cols-2 xl:grid-cols-4; clamp(16px, 4vw, 32px)

Constraint: No data fetching or API changes. Visual + Tailwind only. Run `tsc --noEmit` after.

Part 10 — Engineering Principles (from Architecture Baseline)

These are in effect from this document forward. Every PR must reference the relevant principle.

Principle	Implication
Outcome First	Every new route or service must trace to a business objective, not a technical feature
AI Explains Everything	Every AI-generated recommendation must return: why, evidence, confidence, risk, expected outcome, next action
Human Approval	Any paid action, publishing action, or voice/video action must have an explicit approval gate
Learn Once	Every approved/rejected asset, every experiment result, every review must update Marketing Memory
Progressive Disclosure	No new internal complexity may surface in the founder UI
No Duplication	Before writing a new service, route, or table: search existing codebase. Reuse or extend.

ADR Required	Any new architectural component (table, service, agent, pattern) needs an ADR before merge
Context Engine Gate	No AI call bypasses assembleContext(). This is enforced at PR review.

LaunchMind Blueprint v2.0 · Vijay Tomar · Phoenix, AZ · July 2026 Source: docs/architecture-baseline-v1.md · Current build: CLAUDE.md §11 Stack: Next.js 14 + Vercel · Fastify + Oracle Cloud VM · Supabase · pgvector · BullMQ · Claude API · AWS KMS · Cloudflare